

Avantrade Self Service SDK – Android Integration (Steps 1–6)

1. Integration Flow

1. Prepare SDK configuration
2. Register SDK listeners
3. Launch the SDK with a target page
4. SDK validates configuration and initializes session

All subsequent operations are handled internally by the SDK.

2. SDK Configuration

Create a valid `SelfServiceConfiguration` instance before proceeding.

```
SelfServiceConfiguration config = new SelfServiceConfiguration(  
    AvantradeEnvironment.SIT,           // DEV | UAT | PROD | SIT  
    expiryAtEpochMillis                // long (epoch millis)  
);
```

3. Listener Registration

Listeners **must** be registered **before** calling `AvantradeSDK.launch(...)`.

3.1 State Listener

Observes SDK lifecycle: configuration validation, session init, WebView preload, and error handling. Use it to show loading states or catch errors.

```
AvantradeSDK.setStateListener((state, message) -> {  
    switch (state) {  
        case LOADING:    // Initializing or background work  
            break;  
        case SUCCESS:    // A step completed successfully  
            break;  
        case ERROR:      // SDK stopped due to an error  
            break;  
    }  
});
```

4. Launch SDK

After configuration and listeners are set, launch the SDK with a One Time Token and a target `Page`. The SDK will exchange the One Time Token with Avantrade's backend to obtain a JWT and Refresh Token — after which the token is immediately revoked. This triggers: config validation → token exchange → session init → WebView preload → UI launch.

```
AvantradeSDK.launch(context, configuration, oneTimeToken, page);
```

Parameter	Type	Description
<code>context</code>	<code>Context</code>	Android context
<code>configuration</code>	<code>SelfServiceConfiguration</code>	SDK config (environment + expiry)
<code>oneTimeToken</code>	<code>String</code>	Short-lived token obtained from your backend. Used once to retrieve JWT — immediately revoked after exchange.
<code>page</code>	<code>Page</code>	Target screen to launch (see Section 6)

See **Section 6** for the full list of supported pages and their usage.

5. Disposing the SDK

Call this method when the user logs out or when the SDK is no longer needed. This will release all internal resources, including the preloaded WebView and session state.

After calling `dispose()`, the SDK cannot be used until `launch()` is called again.

 Do not call `dispose()` during normal navigation or app backgrounding, as this will force a full reload on the next launch.

```
AvantradeSDK.dispose();
```

6. Supported Pages

The SDK uses a `Page` abstraction to determine which screen is loaded on launch. Pass a `Page` instance as the **fourth** argument to `AvantradeSDK.launch(...)`.

6.1 DashboardPage

Launches the main dashboard screen. No additional parameters required.

```
AvantradeSDK.launch(context, config, oneTimeToken, new DashboardPage());
```

6.2 InvestmentFundPage

Launches the product holding screen for a specific investment fund type. Requires a `typeId` corresponding to the fund's unique identifier.

Parameter	Type	Description
<code>typeId</code>	<code>String</code>	Unique identifier of the investment fund type

```
AvantradeSDK.launch(context, config, oneTimeToken, new InvestmentFundPage("investment-type"));
```

6.4 Page Summary

Page Class	Target Screen	Parameters
DashboardPage	Main dashboard	None
InvestmentFundPage	Product holding	type (String)